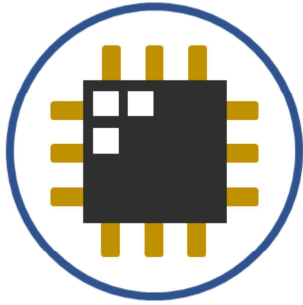
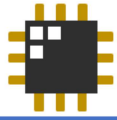




EEE 143: Switching Theory and Digital Logic Design

Lecture 14: Algorithmic State Machines

1



Going towards higher abstractions

Using our current idea of state machines, how do we create
a digital circuit the factorial of a number?

```
1 a = 1
2 b = N
3 while(b != 0):
4     a = a * b
5     b = b - 1
6 return a
```

Before: a = 1, b = 5

Loop 1: a = 5, b = 4

Loop 2: a = 20, b = 3

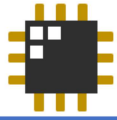
Loop 3: a = 60, b = 2

Loop 4: a = 120, b = 1

Loop 5: a = 120, b = 0

2

In our previous lessons, we've explored how state machines work — primarily using simple logic-based transitions. Now we're extending that idea to solve more complex problems, such as calculating the **factorial of a number**. This challenges us to think beyond basic state transitions and into **structured sequences of operations**, which brings us closer to **higher levels of abstraction** in digital design.



Going towards higher abstractions

Unlike the FSM examples, **the number of different permutations of states & inputs** is prohibitively large in most applications.

*“We must design circuits with a **modular, hierarchical approach**.”*
– Mano, Kime

```
1 a = 1
2 b = N
3 while(b != 0):
4     a = a * B
5     b = b - 1
6 return a
```

Before: a = 1, b = 5

Loop 1: a = 5, b = 4

Loop 2: a = 20, b = 3

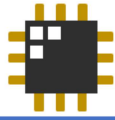
Loop 3: a = 60, b = 2

Loop 4: a = 120, b = 1

Loop 5: a = 120, b = 0

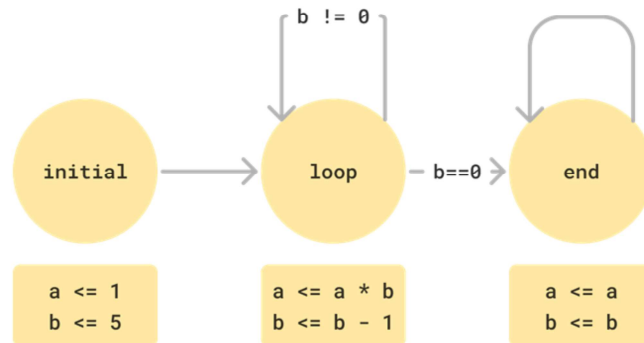
3

As we tackle more realistic applications, we quickly realize that the number of possible **state and input combinations** becomes **too large** to manage manually, as we did with basic FSMs. This complexity demands a better strategy — one that embraces **modularity** and **hierarchical design**, as emphasized by Mano and Kime. In short, we need a way to build complex circuits **from simpler, reusable building blocks**.



What do we need?

```
1  a = 1
2  b = N
3  while(b != 0):
4      a = a * B
5      b = b - 1
6  return a
```



We need to create state machines where, instead of looking at individual bits, we think about the **operations** that need to be performed every cycle

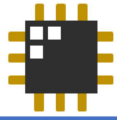
... somewhat like a *flowchart*, essentially.

4

To manage this complexity, we must rethink how we define state machines.

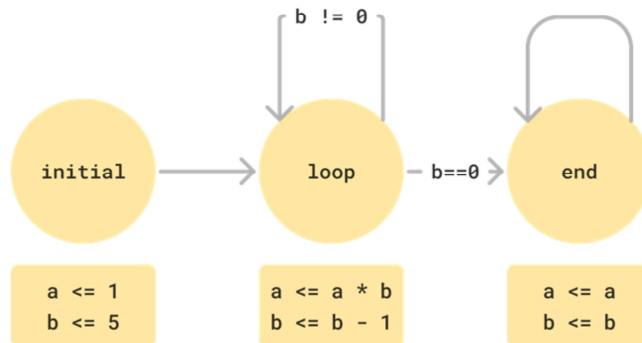
Instead of handling just **individual bits and signals**, we should define states based on the **high-level operations** we want to perform in each clock cycle.

This leads us to a more structured approach — much like creating a **flowchart**, where each block represents a meaningful action rather than low-level logic.



What do we need?

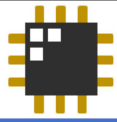
```
1  a = 1
2  b = N
3  while(b != 0):
4    a = a * B
5    b = b - 1
6  return a
```



- We need **integer** memories to hold data ← Registers
- We need to add “**register instructions**” to the state machines:

Algorithmic State Machines (ASM) & Register Transfer Language (RTL)

To support this operational view, we need **registers** to store data — values like counters, results, or loop variables. And more importantly, we need to describe what each state *does* to these registers. This is where **Algorithmic State Machines (ASM)** come in. Paired with **Register Transfer Language (RTL)**, ASM lets us describe both the **control flow** and the **data operations** that occur in each state, allowing for clearer, more scalable digital circuit design.

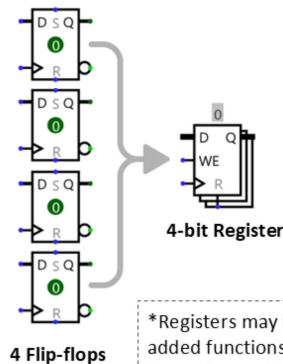


Registers

“We must design circuits with a **modular, hierarchical approach**.” – Mano, Kime

Registers are groups of flip-flops

We will focus on the **transfer and manipulation of data between registers** to abstract things.



We manipulate the data in registers with **combinational blocks (weeks 5 & 6)**.

In our flowcharts, we can note what should be done to a register with the **register transfer language (RTL)**.

Examples of RTL:

$R1 \leftarrow R1 + R2$

$R1 \leftarrow R1 \ll 4$

$R5 \leftarrow 0$

RTL also means **register transfer level**—the level of abstraction we are introducing right now (above logic gates & flipflops)

6

Registers are essential building blocks in digital systems. At their core, **registers are simply groups of flip-flops** used to store multi-bit data, like integers or instruction values. However, they often include additional control features like a **write enable (WE)** signal, which determines when new data should be written into the register.

In this part of the course, our focus shifts toward the **movement and manipulation of data between registers**. Instead of always thinking in terms of individual gates or flip-flops, we now work at a **higher abstraction** — one that deals with **register operations** as the fundamental unit of action.

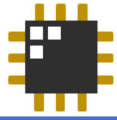
To actually **perform operations on registers**, we use **combinational logic blocks** — the same kind we studied in Weeks 5 and 6 — to do tasks like addition, shifting, or setting a value.

We use **Register Transfer Language (RTL)** in our **flowcharts** and state machine diagrams to specify what actions should occur in a given step or state. RTL gives us a compact, readable way to describe register operations.

Some examples:

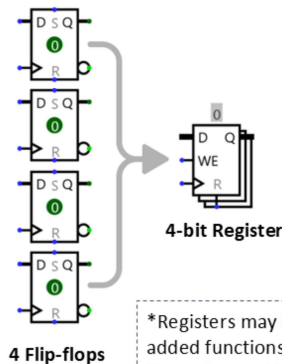
- $R1 \leftarrow R1 + R2$ means register R1 gets the sum of R1 and R2.
- $R1 \leftarrow R1 \ll 4$ means shift the contents of R1 left by 4 bits.
- $R5 \leftarrow 0$ means clear the contents of R5 (set to zero).

Lastly, **RTL** doesn't just refer to the language itself — it also stands for **Register Transfer Level**, which is the **level of abstraction** we're now entering. It sits **above individual logic gates and flip-flops**, allowing us to design more complex systems efficiently by focusing on *what* should happen to data, not *how* it happens at the gate level.



Register notations

Registers are groups of flip-flops



To refer to the bits inside registers, we use **slice notation**.

A is an 8-bit register, we usually refer to it as **A[7:0]**

$A \leftarrow 0x5A$	A[7]	A[6]	A[5]	A[4]	A[3]	A[2]	A[1]	A[0]
	0	1	0	1	1	0	1	0
$A[7:4]$	A[7]	A[6]	A[5]	A[4]				
	0	1	0	1				
$A[4]$				A[4]				
				1				

To merge registers or groups of bits together, we typically write them with curly braces **{}**

A	A[3]	A[2]	A[1]	A[0]		B	B[3]	B[2]	B[1]	B[0]
	1	0	1	0			0	1	0	1
$\{A,B\}$	A[3]	A[2]	A[1]	A[0]	B[3]	B[2]	B[1]	B[0]		
	1	0	1	0	0	1	0	1		
$C \leftarrow \{A,B\}$	C[7]	C[6]	C[5]	C[4]	C[3]	C[2]	C[1]	C[0]		
	1	0	1	0	0	1	0	1		

In digital systems, we often need to access specific bits or groups of bits within a register. To make this easier, we use **slice notation**. For example, if we have an 8-bit register A, we refer to it as A[7:0], where A[7] is the most significant bit (MSB) and A[0] is the least significant bit (LSB). This notation helps us clearly indicate which part of the register we're working with, especially when performing bit-level operations.

Example:

If $A \leftarrow 0x5A$, then in binary:

$A[7:0] = 01011010$

Using slice notation:

- $A[7:4] = 0101$
- $A[4] = 1$

Sometimes, we also need to **combine multiple registers or groups of bits** into one. This is called **concatenation**, and it is written using curly braces **{}**. Concatenation is useful when building larger data words or formatting data in a specific way for processing.

Example:

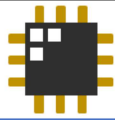
Let $A[3:0] = 1010$ and $B[3:0] = 0101$

Then $\{A, B\} = 10100101$

If stored in register C, we write:

$C \leftarrow \{A, B\}$ so that $C[7:0] = 10100101$

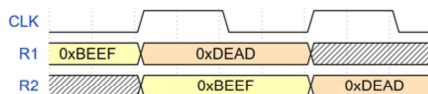
By using slice notation and concatenation, we gain precise control over data inside our registers. These tools are especially important at the **Register Transfer Level (RTL)**, where we describe what happens to data between registers in each clock cycle—providing a higher-level view than just logic gates and flip-flops.



Register Transfer Language (RTL)

- All operations/assignments are performed upon the positive edge (posedge) of CLK

- e.g. for $R2 \leftarrow R1$...



Recall the basic flip-flop functionality:
Inputs are checked before the positive edge,
 and **output updates after** the positive edge

Register Transfer	Description
$R2 \leftarrow R1$	Transfer/Copy contents from R1 to R2
If (A = 1) then ($R2 \leftarrow R1$)	Conditional statement. A is an input literal called a "control signal" .
$R3 \leftarrow R1 + R2$	Add contents of R1 to R2 and place result into R3
$R3 \leftarrow R3 + 1$	Increment R3
$R4 \leftarrow R4 \ll 2$	Shift bits of R4 to the left 2 times
$R5 \leftarrow 0$	Clear/Reset R5
$R6 \leftarrow \{R3, R4\}$	Concatenate R3 and R4 (R6 must have more bits)

8

In Register Transfer Language (RTL), **all operations and assignments occur on the positive edge (posedge) of the clock signal (CLK)**. This reflects how real digital circuits work: data inputs are evaluated just before the rising edge of the clock, and the register values are updated right after that edge. This synchronization ensures consistent timing and predictable behavior in digital systems.

For example, when we write $R2 \leftarrow R1$, we are describing a register transfer that happens on the next positive clock edge. This statement means that the contents of R1 will be copied to R2.

Example:

$R2 \leftarrow R1$

→ Transfers the contents of R1 into R2.

We can also add control logic using conditional statements. A common structure is using an input signal—often called a **control signal**—to determine if a register operation should happen.

Example:

If (A = 1) then ($R2 \leftarrow R1$)

→ Only transfer R1 to R2 if control signal A is high.

More complex operations can also be expressed using RTL, such as arithmetic or logical operations between registers.

Examples:

$R3 \leftarrow R1 + R2$

→ Adds the contents of R1 and R2, storing the result in R3.

$R3 \leftarrow R3 + 1$

→ Increments the value of R3 by 1.

$R4 \leftarrow R4 \ll 2$

→ Shifts the bits in R4 to the left by 2 positions.

$R5 \leftarrow 0$

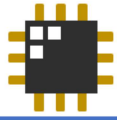
→ Clears or resets the register R5 by setting all bits to zero.

$R6 \leftarrow \{R3, R4\}$

→ Concatenates the values of R3 and R4, storing the combined result in R6.

Note that R6 must have enough bits to hold the total number of bits from R3 and R4 combined.

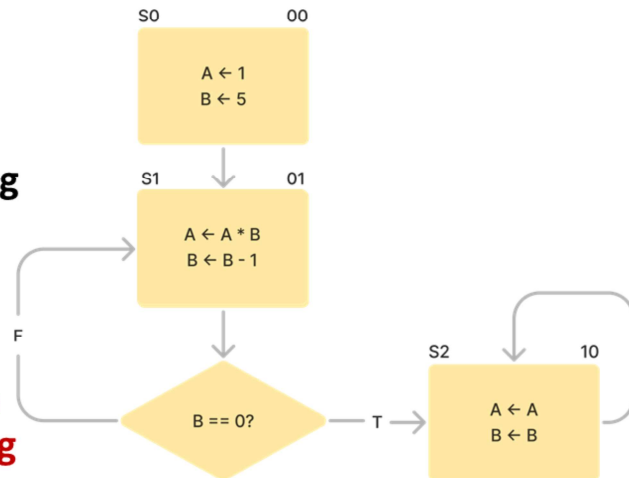
This structured and clocked approach using RTL allows us to precisely describe the behavior of digital systems at the **register transfer level**, making it easier to design, simulate, and implement complex hardware circuits.



Algorithmic State Machines (ASMs)

- ASMs are like coding flowcharts, but they contain detailed implications about **the registers, states & timing**
- Powerful modelling tool when combined with **RTL**

We'll use this to create more complex systems by describing the abstract behavior



9

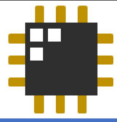
Algorithmic State Machines (ASMs) are a powerful way to represent the behavior of digital systems. They are similar to flowcharts used in programming, but with a key difference: ASMs include **explicit details about registers, control logic, and timing**. This means that every operation, decision, and transition in an ASM is directly linked to what the hardware should do during each clock cycle.

When combined with **Register Transfer Language (RTL)**, ASMs become a powerful modeling tool. RTL describes how data moves and transforms between registers, while the ASM outlines the sequencing and control of those actions. Together, they allow us to **design and visualize complex digital systems** in a structured and hierarchical way.

We'll use ASMs to model more sophisticated operations, where simple finite state machines (FSMs) are not enough. They help us **abstract the behavior** of a circuit without getting lost in low-level gate logic.

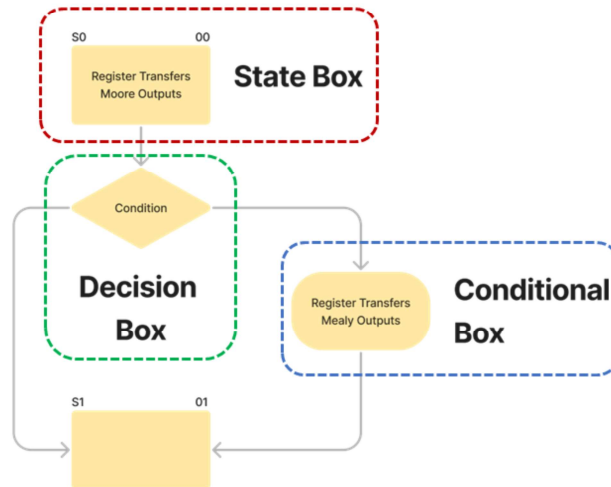
On this slide, there is an ASM diagram that shows how the factorial of a number can be computed. This example illustrates how ASM diagrams capture the

step-by-step logic, register updates, and control flow needed for computation.



Parts of an ASM

- FSM states turn into **state boxes**
- **Decision boxes** allow if/else and loops
- **Conditional boxes** do things after a condition box, but it is not a “state”



10

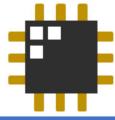
An **Algorithmic State Machine (ASM)** is composed of specific elements that extend the classic finite state machine (FSM) to better describe complex operations. First, the **FSM states** are represented as **state boxes**, which are shown as rectangles. These indicate the main control states of the system and often include register operations that occur in that state.

Next, **decision boxes**—shaped like diamonds—are used to evaluate conditions, similar to if-else statements in programming. They allow the ASM to handle branching and loops by deciding which path the system should follow based on inputs or internal values.

Finally, **conditional boxes**, drawn as rounded rectangles, are placed after decision boxes. These boxes specify operations that are executed only if a certain condition is met, but they are not considered actual states. This separation of decisions and actions makes ASMs more expressive and better suited for describing detailed behaviors.

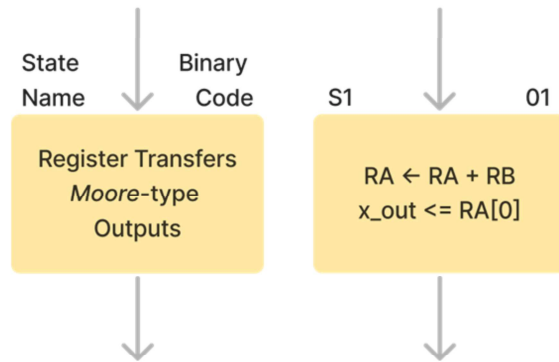
Together, these parts allow ASMs to model control flow, logic, and data manipulation in a visual and structured way—making them especially useful

when designing complex digital circuits.



ASMs : State Box

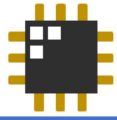
- “Moore Box”
- Contains the **state name** on the upper left, **binary code** (state assignment) on the upper right
- Moore-type outputs are assigned here



11

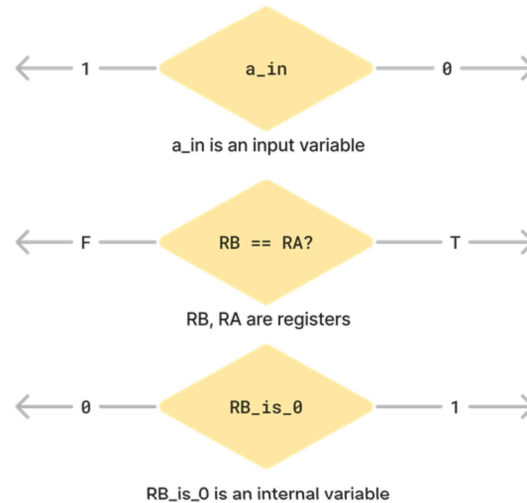
In an **Algorithmic State Machine (ASM)**, the **State Box**, often referred to as a **Moore Box**, represents one specific control state of the system. It is drawn as a **rectangle** and contains important information. On the **upper left corner**, you'll find the **state name** (e.g., S0, Init, Calc, etc.), while the **upper right corner** usually shows the **binary code** or **state assignment** for implementation purposes. Inside the box, you write the **Moore-type outputs**, which are outputs or register operations that happen unconditionally while the system is in that state. Since this follows the Moore model, these outputs depend only on the current state and not on the inputs.

This box forms the core structure of the ASM diagram and defines what actions the system takes during each clock cycle when it is in that particular state.



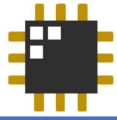
ASMs : Decision Box

- Reflects **if/else** logical statements
- Can use input literals, registers, or internal combinational signals for the decision



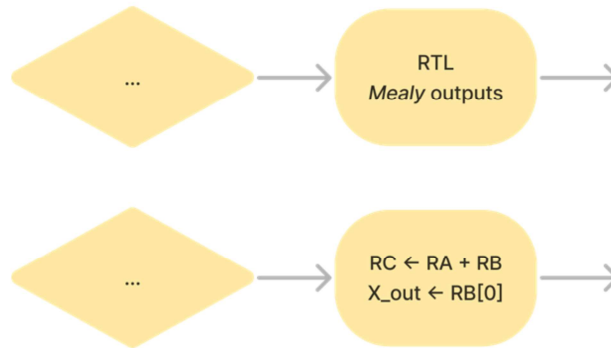
12

The **Decision Box** in an **Algorithmic State Machine (ASM)** represents a point where the control flow branches based on a condition—much like an if/else statement in programming. It is drawn as a **diamond-shaped box** and evaluates a **logical expression** to determine the next path in the diagram. The condition inside can involve **input literals** (like external signals), **register values**, or even **internal combinational logic signals**. Based on whether the condition is true or false, the ASM will follow different outgoing paths from the decision box. This structure enables loops, conditional execution, and branching, making it a vital part of expressing complex control behavior in digital systems.



ASMs : Conditional Box

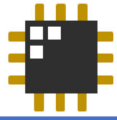
- “Mealy Box”
- Input must come from a condition box (because why else do you need it)
- Mealy outputs are assigned here



Recall Mealy machine behavior:
Output changes depending on the input condition

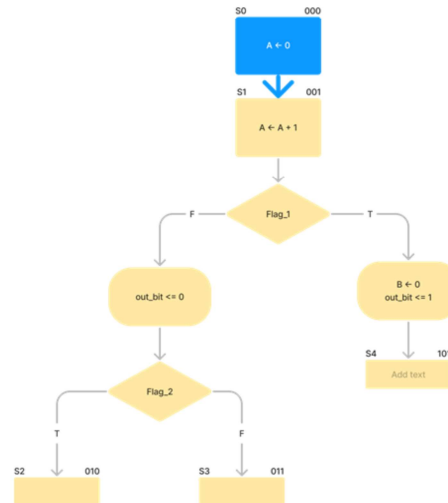
13

The **Conditional Box** in an **Algorithmic State Machine (ASM)** is often called a **Mealy Box**, because it reflects the behavior of a Mealy machine. It is drawn as a **rounded rectangle** and is always placed **after a Decision Box**, since it only activates when a specific condition is true. Inside this box, we assign **Mealy-type outputs**—that is, outputs or register operations that depend on both the current state and the input condition. This allows for more responsive behavior in a digital system, where certain actions occur only when a condition is met during a specific state. In contrast to the Moore box (where outputs depend only on the state), the Mealy box allows outputs to change immediately based on input values.



ASMs : State Transition Conventions

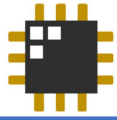
- At any point in time, we are in one of the **state boxes**.
(because it's a state)
- Register transfer instructions inside the state boxes are performed **at the next clock cycle**.
- Here, $A \leftarrow 0$ will be performed as we transition to S1.



14

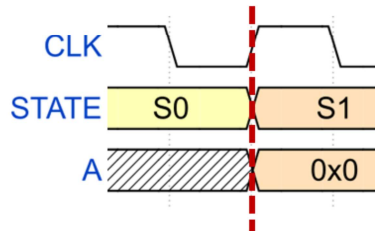
In an **Algorithmic State Machine (ASM)**, the system is always in **exactly one state box at any given time**—this reflects the current control state of the system. The transitions between these state boxes are determined by the conditions in decision boxes, and possibly followed by actions in conditional boxes.

One important convention is that **any register transfer instruction inside a state box is executed on the next clock cycle**, right as the system transitions into that state.



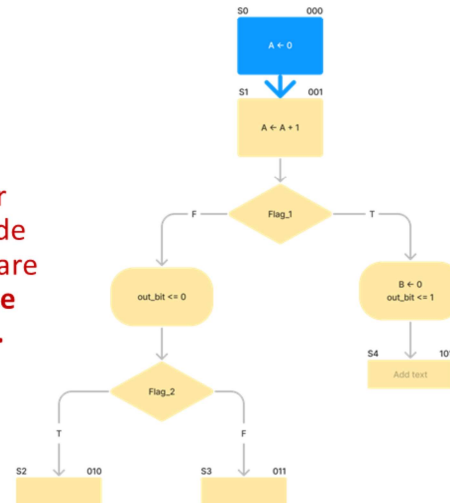
ASMs : State Transition Conventions

- At any point in time, we are in one of the **state boxes**.
(because it's a state)



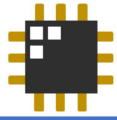
- Register transfer instructions inside the state boxes are performed **at the next clock cycle**.

- Here, $A \leftarrow 0$ will be performed as we transition to S1.



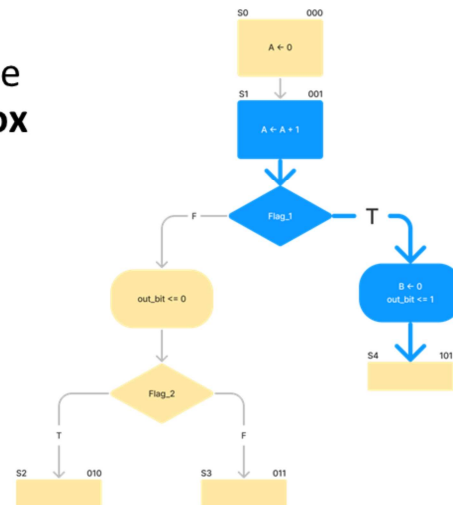
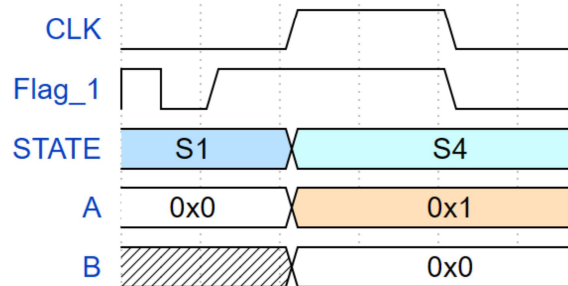
15

For example, if we see an instruction like $A \leftarrow 0$ inside the state box labeled S1, it means that A will be set to 0 **as the system enters S1**, during the positive edge of the clock. This ensures precise timing and control over when and how operations are executed, aligning with synchronous digital circuit behavior.



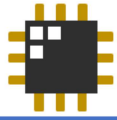
ASMs : State Transition

- We are set to perform all of the “instructions” from the current state box to **just before the next state box**



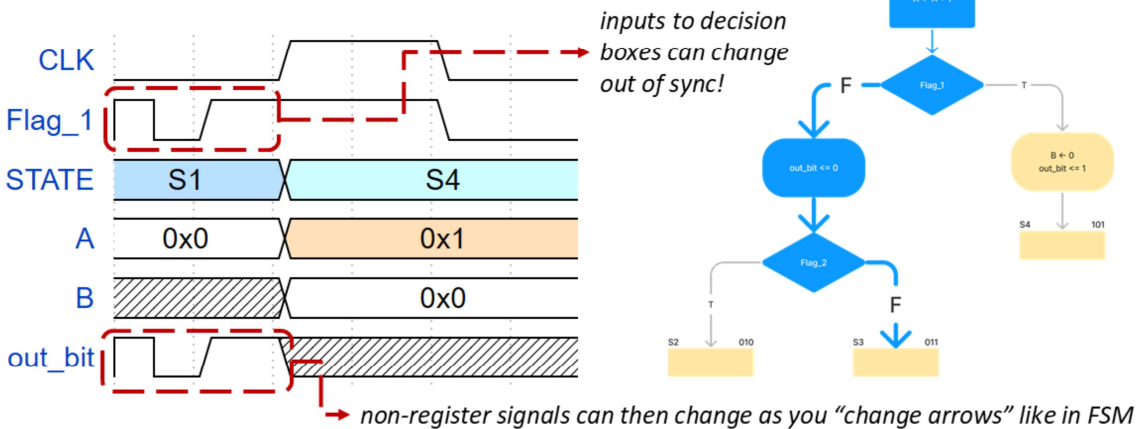
16

During the transition, the system is set to **execute all the instructions defined from the current state box up to—but not including—the next state box**. This includes any actions written inside the state box itself (which are triggered on the clock edge), as well as operations specified in conditional boxes that follow decision logic. This structure ensures that every transition represents a clearly defined sequence of operations, maintaining synchronization and predictability in digital circuit behavior. It emphasizes that the state box defines *where you are*, while the logic and operations between state boxes define *what gets done* as you move between them.

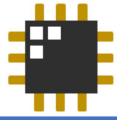


ASMs : State Transition

- Changes to the inputs of decision boxes cause changes in the “**path to take**”

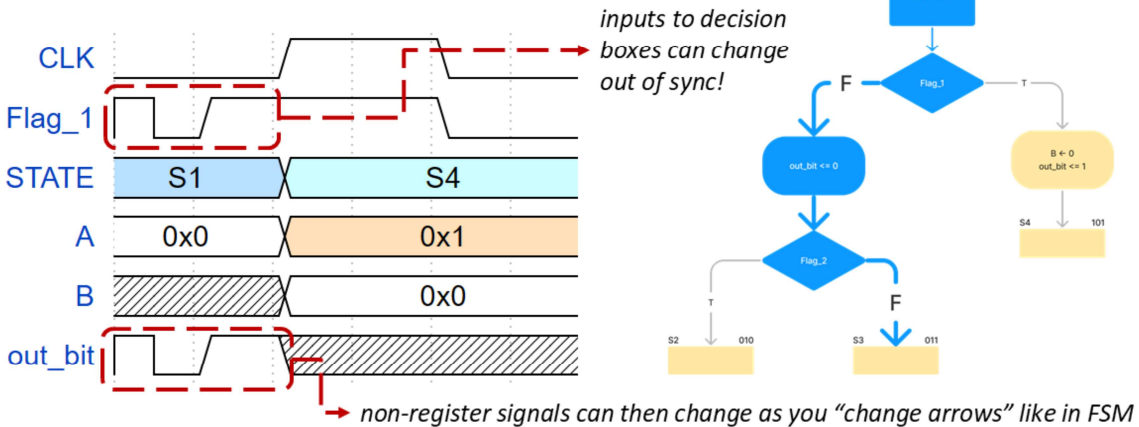


In **Algorithmic State Machines (ASMs)**, the **inputs to decision boxes** play a critical role in determining the **path the system will take** through the state diagram. These inputs can come from various sources—such as control signals, register values, or other combinational logic—and they may **change independently of the clock**. This means that decision box inputs can update **out of sync** with the system’s regular state transitions.

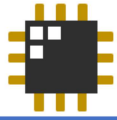


ASMs : State Transition

Non-registers in **conditional boxes** can change immediately

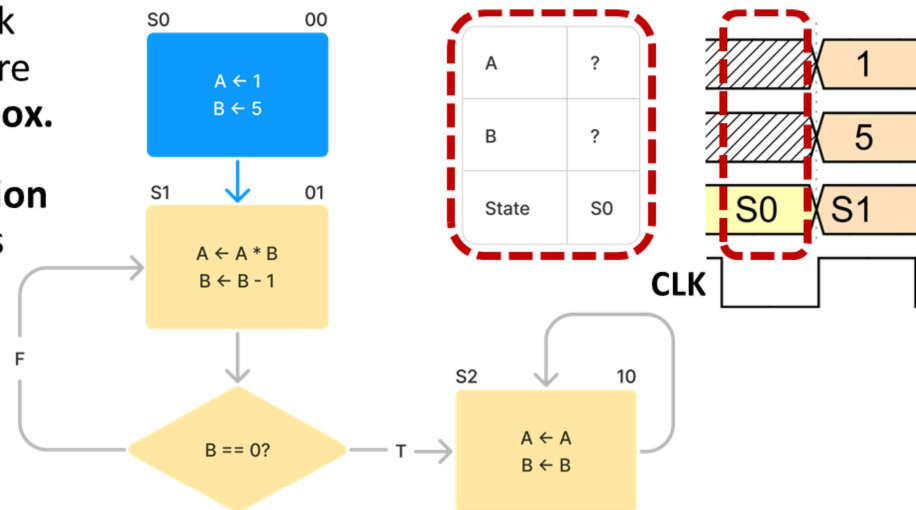


As a result, the ASM may **change direction or behavior "on the fly"** as these input conditions vary, much like how transitions in a classic FSM (Finite State Machine) are influenced directly by inputs. Since these inputs are often **non-register signals**, they can fluctuate rapidly, causing the ASM to switch paths even between clock cycles. This highlights the need to carefully manage timing and control in ASM design to ensure stable and predictable behavior.



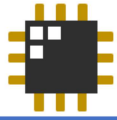
ASM Example: Factorial (Cycle 0)

- At any clock cycle, we are in a **state box**.
- We **transition** state boxes every cycle



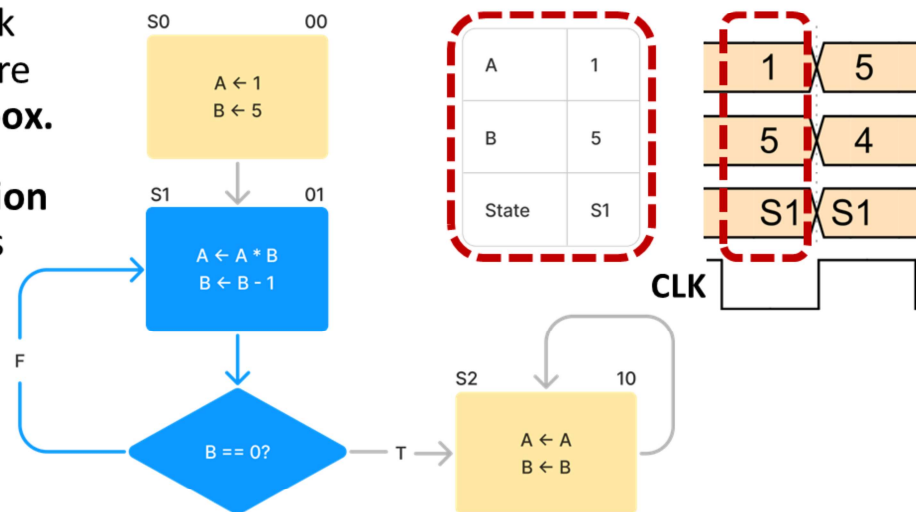
19

At the beginning of the execution, the system is in **State S0**, and both registers **A** and **B** are uninitialized or in an undefined state. This is the idle or reset state where no operations have been performed yet.



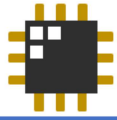
ASM Example: Factorial (Cycle 1)

- At any clock cycle, we are in a **state box**.
- We **transition** state boxes every cycle



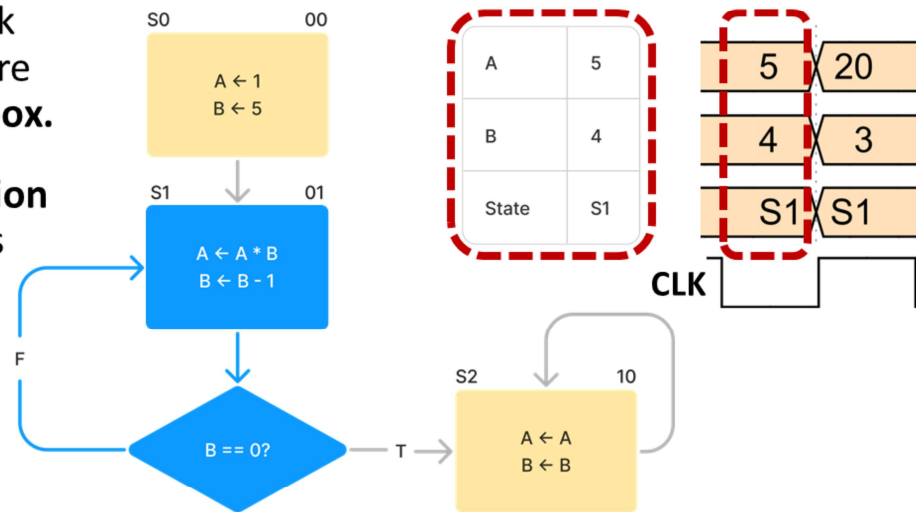
20

Upon transitioning into **State S1**, initial values are loaded into the registers. **A is set to 1**, which acts as the running product accumulator, and **B is set to 5**, representing the number whose factorial will be computed. This sets up the loop for multiplication.



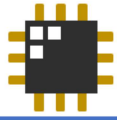
ASM Example: Factorial (Cycle 2)

- At any clock cycle, we are in a **state box**.
- We **transition** state boxes every cycle



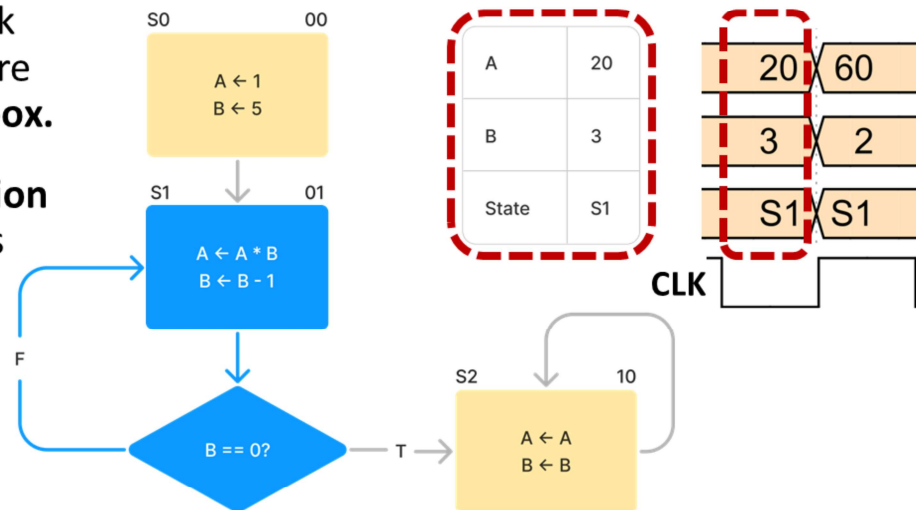
21

Now that the loop has started, the first operation is executed: **A becomes $A \times B = 1 \times 5 = 5$** , and **B is decremented from 5 to 4**. The ASM stays in **State S1**, repeating this loop as long as B is not zero.



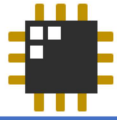
ASM Example: Factorial (Cycle 3)

- At any clock cycle, we are in a **state box**.
- We **transition** state boxes every cycle



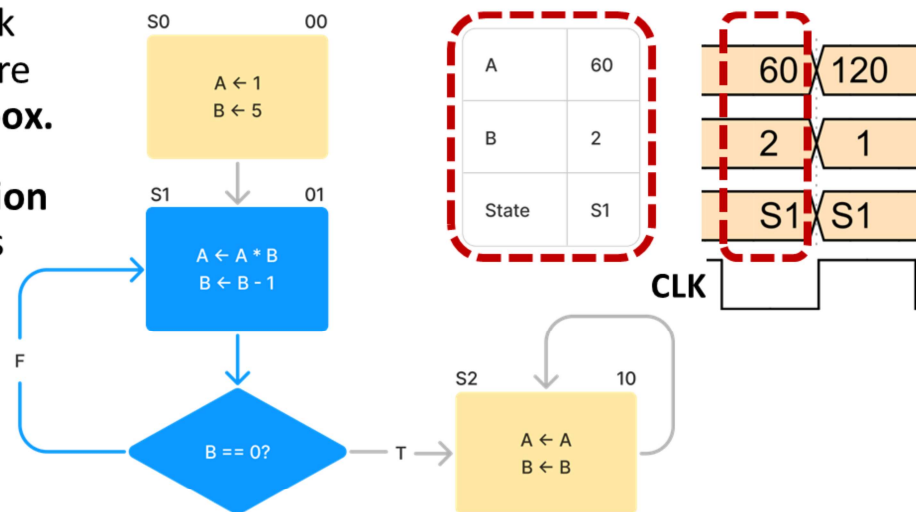
22

In the next iteration, $A = 5 \times 4 = 20$, and **B is decremented to 3**. The loop continues, maintaining the factorial computation with updated values.



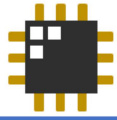
ASM Example: Factorial (Cycle 4)

- At any clock cycle, we are in a **state box**.
- We **transition** state boxes every cycle



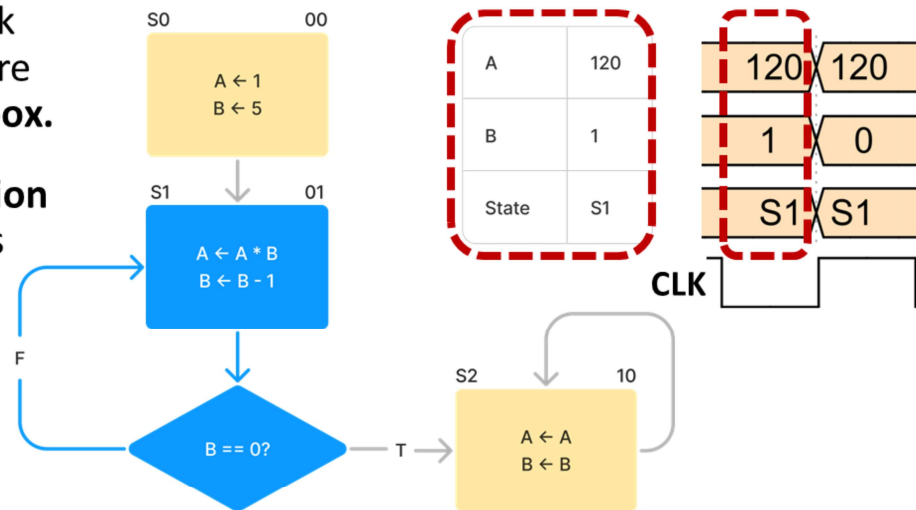
23

Now, $A = 20 \times 3 = 60$, and **B is reduced to 2**. Each cycle repeats the same structure: multiply A by B, then decrement B.



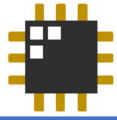
ASM Example: Factorial (Cycle 5)

- At any clock cycle, we are in a **state box**.
- We **transition** state boxes every cycle



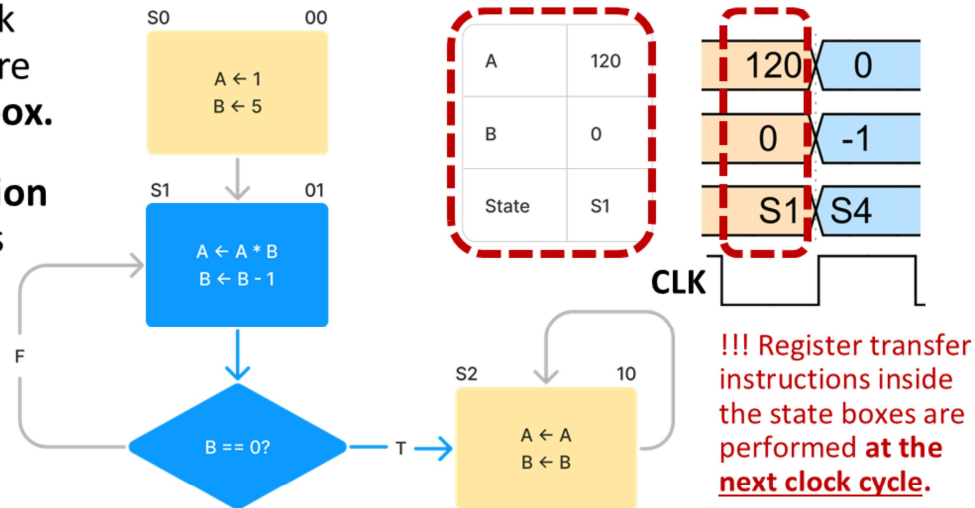
24

In this iteration, $A = 60 \times 2 = 120$, and **B becomes 1**. We're nearing the end of the loop as B approaches zero.



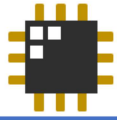
ASM Example: Factorial (Cycle 6)

- At any clock cycle, we are in a **state box**.
- We **transition** state boxes every cycle



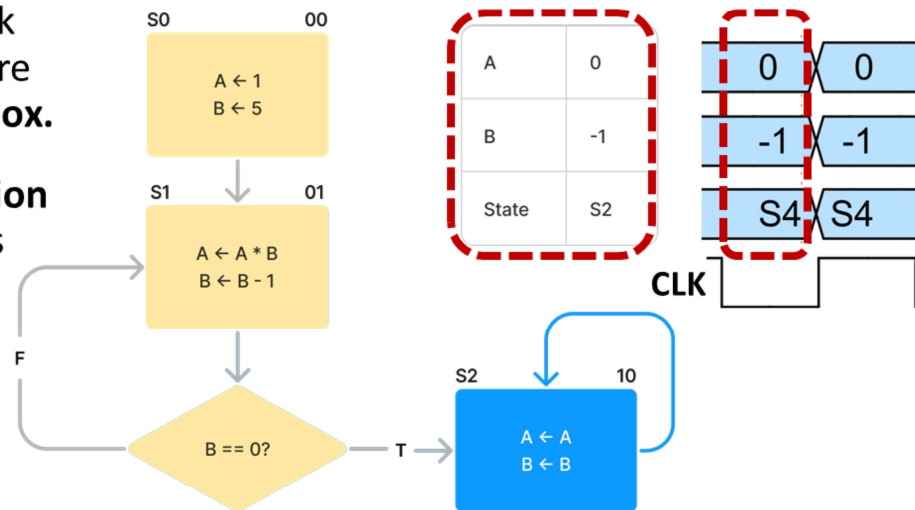
25

At this point, the registers still hold **A = 120** and **B = 0** *before* the clock edge. The multiplication $A \leftarrow A * B$ is *scheduled* to happen at the next positive clock edge, so although B is zero now, the operation will be executed one more time, which leads to **A being updated to 0 after the clock edge** (in the following cycle). This means the loop condition ends here, but the register transfer causes one extra multiplication that affects the result.



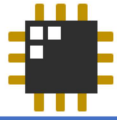
ASM Example: Factorial (Cycle 7)

- At any clock cycle, we are in a **state box**.
- We **transition** state boxes every cycle



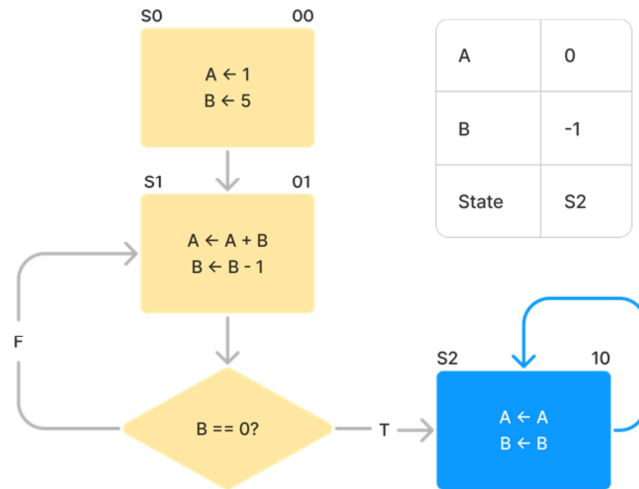
26

The ASM transitions into **State S2**, and at this point, the register **A** is **observed as 0**, while **B** is **-1**. This output is different from what we might expect ($A = 120$ from the previous cycle).



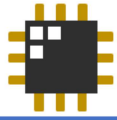
ASM Example: Factorial

- The output is different! (**A = 0** vs **A = 120**)
- This is because in ASMs, we perform the **register transfers in the current state at the next posedge CLK**
- Hence, $A \leftarrow A * B$ was executed once more than necessary



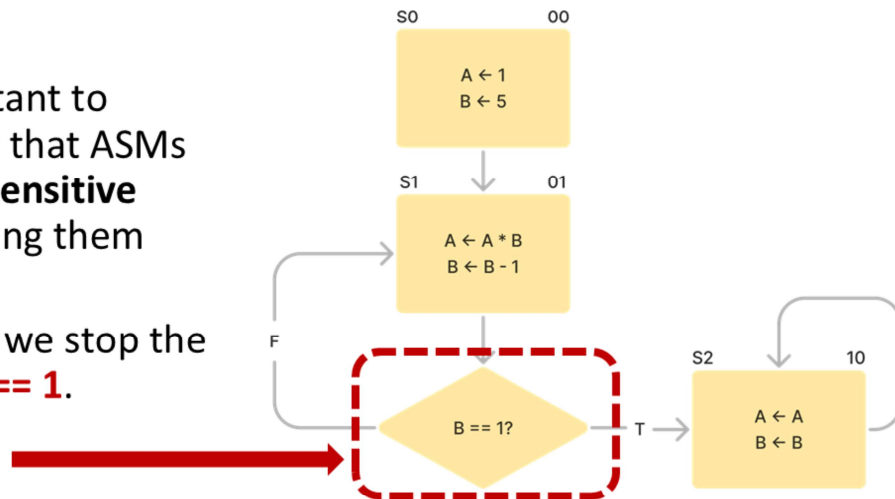
27

The reason for this discrepancy lies in how ASMs handle register transfers: all register updates defined in a state are actually performed at the **next positive edge of the clock**. As a result, the operation $A \leftarrow A * B$ was executed **one cycle too many**, causing the factorial value in A to be overwritten with 0. This highlights the importance of carefully timing register transfers in ASM design to avoid unintended extra computations.



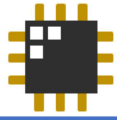
ASM Example: Factorial

- It is important to remember that ASMs are **cycle-sensitive** when writing them
- To fix this, we stop the cycle at **B == 1**.

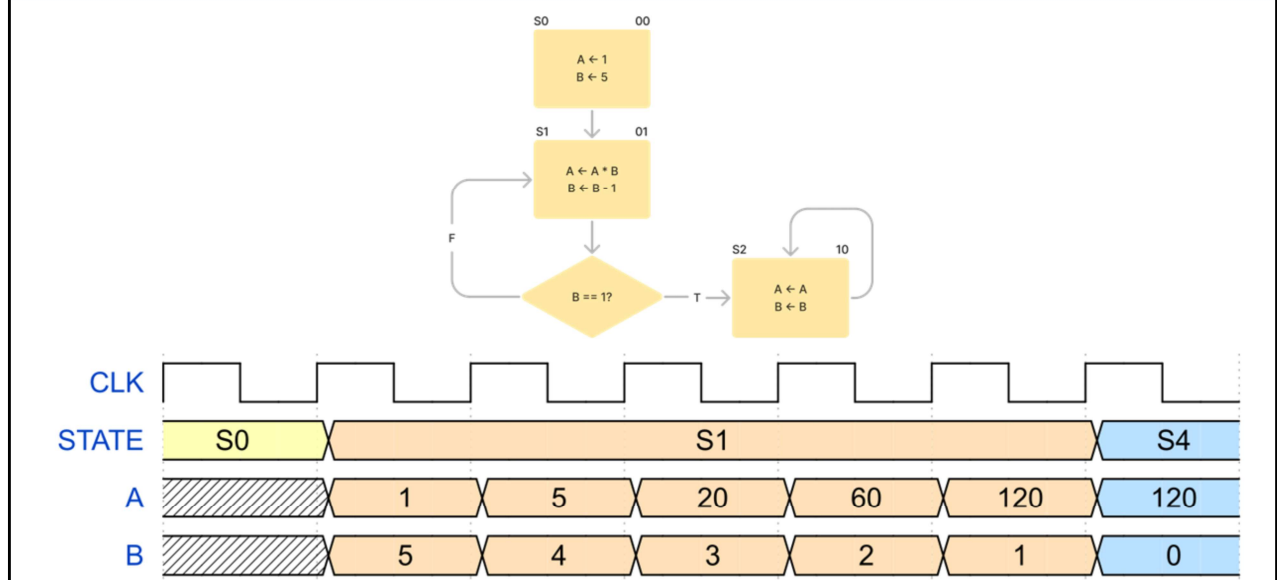


28

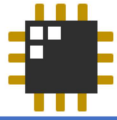
It is important to remember that **ASMs are cycle-sensitive**, meaning that all register transfers inside a state execute at the *next* clock edge. Because of this, operations like $A \leftarrow A * B$ may happen one cycle too many, causing incorrect outputs—as we saw when the factorial calculation ran the multiplication with $B = 0$. To fix this, the ASM design should **stop the loop when $B == 1$ instead of $B == 0$** . This adjustment prevents the extra multiplication cycle, ensuring the result in register A remains correct and avoids unintended updates after the factorial is fully computed.



ASM Example: Factorial



The ASM diagram and the clock/timing diagram together demonstrate how the algorithm is implemented in hardware through state transitions and register transfers synchronized to the clock. The ASM shows the logical flow of states, decisions, and data operations, while the timing diagram reveals the precise moments when register values update at each clock's positive edge. This highlights the critical cycle-sensitive nature of ASMs, emphasizing that operations scheduled in one state only take effect after the clock edge, which can lead to extra or delayed updates if not carefully designed. Understanding this interplay between ASM flow and clock timing is essential to correctly implement and debug digital circuits based on algorithmic state machines.

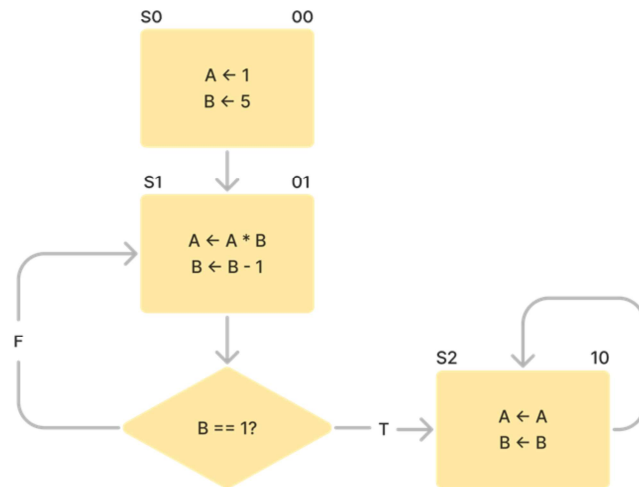


Hardware for ASMs

The ASMs tell us:

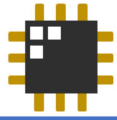
- **what** things need to be done to the registers
- **when** things need to be done to the registers

How do we make circuits for this?



30

Now that we've seen how ASMs describe the behavior of a system at a high level, we can start thinking about how to actually implement that behavior in hardware. The ASM diagrams tell us **what operations need to be performed on the registers**—such as additions, shifts, and transfers—and **when these operations should occur**, based on the system's state and inputs. However, ASMs are only abstract representations. The next step is figuring out how to build circuits that follow this behavior precisely. This brings us closer to the concept of splitting the system into two parts: one that handles the data operations, and one that manages control—something we'll explore next.

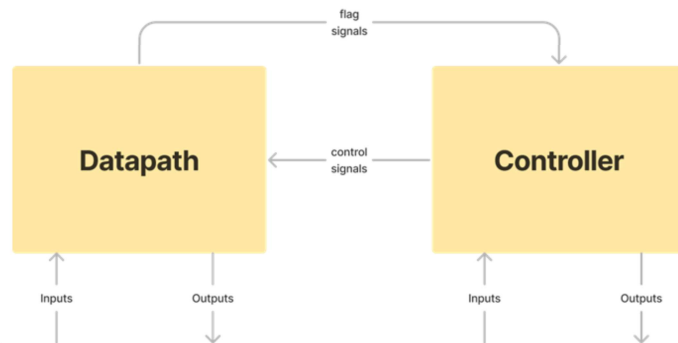


Datapath-Controller Systems

ASMs turn into **datapath-controller systems** as circuits.

The **datapath** contains the registers, the combinational blocks to operate on those registers, and the muxes.

The **control** tells the datapath which register transfer operations to do at any given moment.



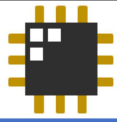
31

When we move from ASM diagrams to actual hardware implementations, we transition into **datapath-controller systems**. These systems are the direct circuit-level realization of the behavior described in an ASM.

The **datapath** is responsible for holding and manipulating data. It includes the **registers** that store values, the **combinational logic blocks** that perform operations like addition, shifting, or comparisons, and the **multiplexers (muxes)** that help route data to the correct destinations within the system.

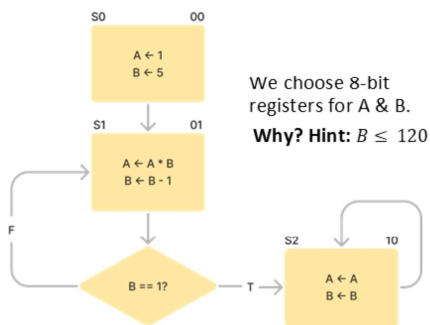
Meanwhile, the **controller**—derived from the ASM’s state transitions—decides **which operations should occur and when**. It sends out control signals that tell the datapath what actions to perform during each clock cycle. For example, it decides whether to load a new value into a register, select a specific input through a mux, or enable a particular arithmetic operation.

This separation of responsibilities allows for a more modular and manageable design, where the datapath focuses on executing operations and the controller ensures those operations happen in the correct sequence, just like what was described in the ASM.



Factorial Example Continued

The **datapath** contains *what* we need to do to the registers.
Hence, we have a multiplier, a subtractor, constants, and MUXes.



32

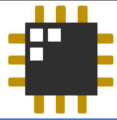
Continuing from the ASM of the factorial example, we now focus on building the corresponding **datapath**. The datapath is responsible for carrying out the actual operations described in the ASM, so it includes all the necessary components to manipulate the registers involved.

In this case, we need to compute the factorial by repeatedly multiplying two values and decrementing one of them. Therefore, our datapath must contain a **multiplier** (to compute $A \times B$), a **subtractor** (to compute $B - 1$), and **constant sources** such as 1 (for decrementing) and 0 (for initialization). We also include **multiplexers (MUXes)** to control which values are loaded into the registers depending on the state of the system.

For storage, we use **8-bit registers** for both A and B. This is a practical choice because we know that the maximum value we expect for the factorial operation is 120 (i.e., $5! = 120$). Since 8-bit registers can represent values up to 255, they are more than sufficient to store any intermediate or final result in our factorial computation.

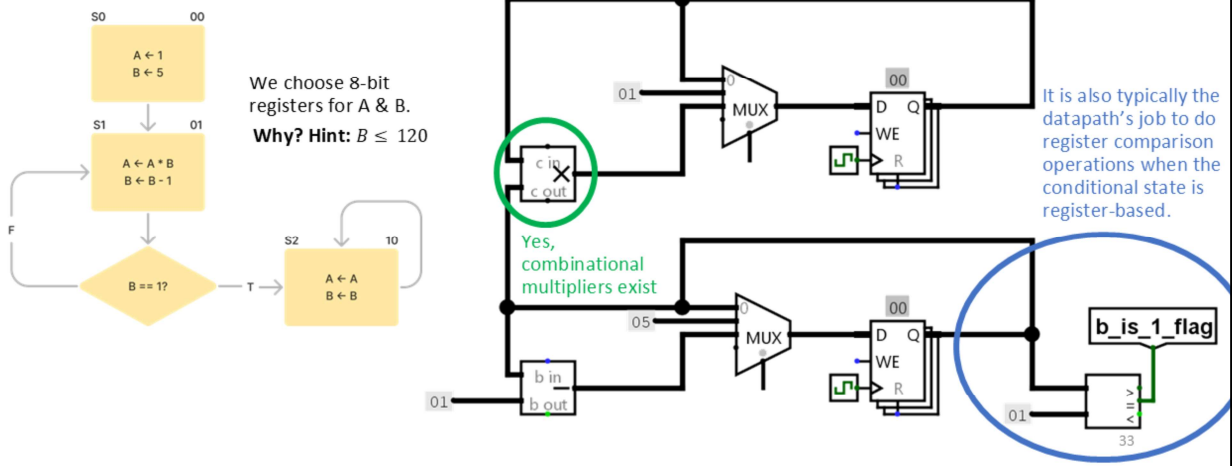
Overall, this datapath reflects the "what" in the ASM: the actual arithmetic and

data movement operations needed to compute the factorial.



Factorial Example Continued

The **datapath** contains *what* we need to do to the registers.
Hence, we have a multiplier, a subtractor, constants, and MUXes.



Let's break down each component of the datapath circuit shown, which implements the factorial example from our ASM diagram.

We begin by identifying the two main registers: the **top register is for A**, and the **bottom register is for B**. These will hold the intermediate and final values involved in computing the factorial.

Register A:

The register for A has **three possible inputs**, selected via a multiplexer:

- 1. The previous value of A** – This allows the register to retain its current value when no change is needed.
- 2. The constant 1** – This is used to initialize A to 1 at the beginning of the computation.
- 3. The product of the previous A and previous B** – This is the core operation for computing the factorial, where A accumulates the product of decreasing values of B.

Register B:

Likewise, the register for B also has **three possible inputs**, selected by another

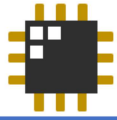
multiplexer:

1. **The previous value of B** – Similar to A, this allows B to remain unchanged if necessary.
2. **The constant 5** – This is used to initialize B with the value of N, the input for the factorial (in this example, 5).
3. **The result of B - 1** – This enables the countdown loop, reducing B by 1 each cycle to eventually terminate the factorial operation.

Comparator:

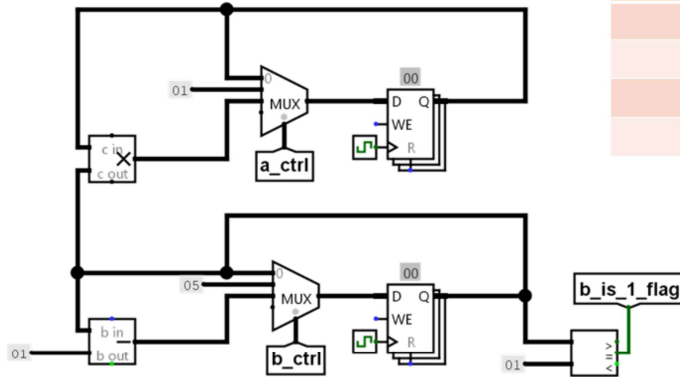
Located at the **bottom right** of the datapath is a **comparator**. Its purpose is to check whether **B is equal to 1**. This comparison is important because it acts as a **decision point** in the ASM—it determines when the multiplication loop should stop and allows the system to transition to the final state.

Together, these components form a working datapath that can follow the instructions given by the controller to carry out the factorial computation in hardware.



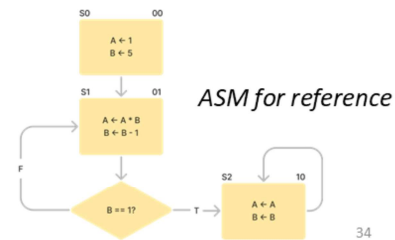
Factorial Example Continued

We still need to tell the datapath *when* to do what things, and this is where **control** comes in. *Control* contains the states! It importantly gives the **MUX SELECTS**.



Let's describe it using a state table!

State	b_is_1	Next State	a_ctrl	b_ctrl

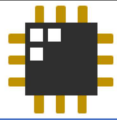


34

In our factorial example, the **datapath** contains all the elements necessary to perform operations—registers, multipliers, subtractors, and multiplexers (MUXes). However, the datapath **doesn't decide on its own when to do what**. That's where the **control** component comes into play.

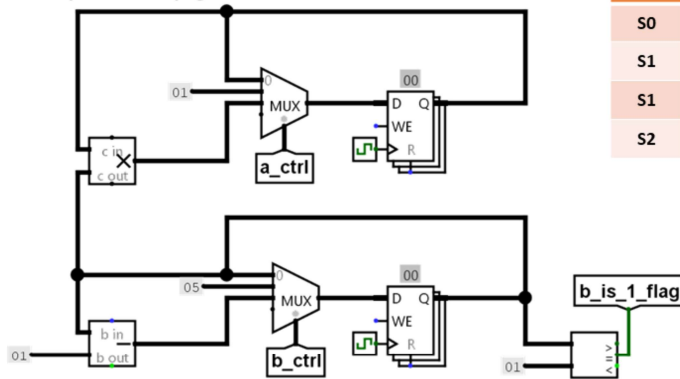
The **control unit** is responsible for sequencing and decision-making. It holds the **states** we defined in our ASM (Algorithmic State Machine), and it ensures that at each clock cycle, the correct operations are triggered. Most importantly, it **determines the select lines for the multiplexers** in the datapath. These **MUX SELECTS** are what enable the datapath to pick between different inputs at the right time—for example, whether A should load a constant 1, retain its value, or compute $A \times B$.

In short, while the datapath does the actual work, the **control tells it what to do and when**, making sure everything runs according to the ASM we've designed.



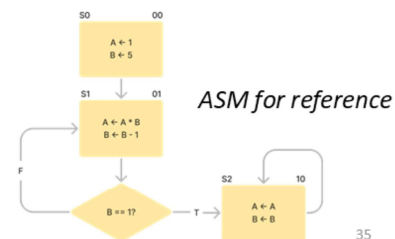
Factorial Example Continued

We still need to tell the datapath *when* to do what things, and this is where **control** comes in. *Control* contains the states! It importantly gives the **MUX SELECTS**.



Let's describe it using a state table!

State	b_is_1	Next State	a_ctrl	b_ctrl
S0	x	S1	01 (choose 1)	01 (choose 5)
S1	0	S1	10 (choose A*B)	10 (choose B-1)
S1	1	S2	10 (choose A*B)	10 (choose B-1)
S2	x	S2	00 (choose A)	00 (choose B)



35

To better understand how control works in our factorial example, we can describe it using a **state table**. This table shows how the control logic determines the behavior of the multiplexers (MUXes) for both **register A** and **register B**, based on the current **state** and the **b_is_1** flag from the comparator.

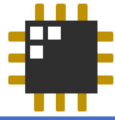
Starting with **State S0**, this is the initialization state. It always transitions to **State S1**, and in this state, we want register **A** to load **1** and **B** to load **5**, which are both constants. These constants are connected to input line 01 of their respective MUXes. Since there's no decision-making here, the **b_is_1** flag is marked as **don't care (x)**—it doesn't affect the control behavior at this point.

Next, we move to **State S1**, which has two possible transitions. If **B** is **not equal to 1**, we remain in **State S1**. In this case, the circuit must update register **A** to **A × B** and register **B** to **B – 1**. These operations come from the output of the multiplier and subtractor, connected to **input line 10** of the MUXes. So, when **b_is_1 = 0**, we keep looping through **State S1** with those register operations.

If instead **B** is **equal to 1**, we move to **State S2**, but we still perform the same

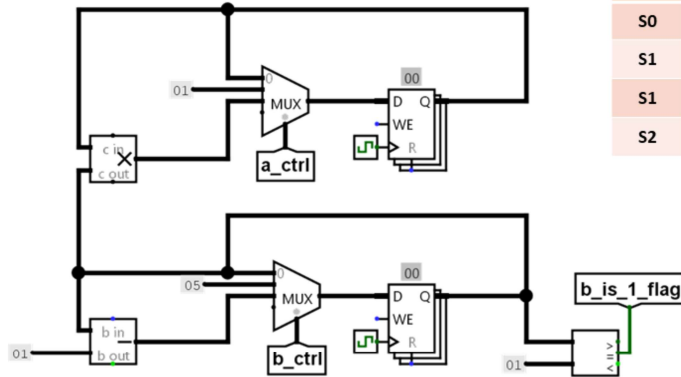
operations: $A \times B$ and $B - 1$, using input 10. The only difference here is that **b_is_1 = 1**, which triggers the transition to the final state. This shows how the control unit uses input flags to guide the flow of states and select the appropriate MUX inputs.

Finally, in **State S2**, we've completed the factorial computation. There's no need to modify the registers anymore. So, we simply hold their previous values by selecting **input 00** of the MUXes, which feed back the current values of A and B. Again, **b_is_1 is marked as 'x'** here since no branching decisions are made in this state.



Factorial Example Continued

We still need to tell the datapath *when* to do what things, and this is where **control** comes in. *Control* contains the states!
It importantly gives the **MUX SELECTS**.



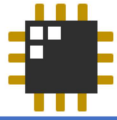
Let's describe it using a state table!

State	b_is_1	Next State	a_ctrl	b_ctrl
S0	x	S1	01 (choose 1)	01 (choose 5)
S1	0	S1	10 (choose A*B)	10 (choose B-1)
S1	1	S2	10 (choose A*B)	10 (choose B-1)
S2	x	S2	00 (choose A)	00 (choose B)

We figure out the next state and the control signals from the ASM and the muxes

36

This state table effectively captures the **decision logic for controlling the datapath** and demonstrates how the controller uses state information and condition flags to guide register operations.

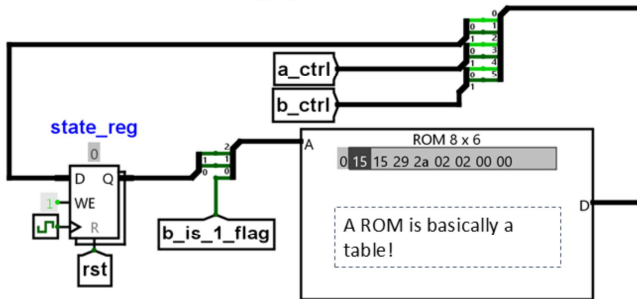


ROM Implementation of Controller

We can do the **usual synthesis** from here to make the controller and connect it to the datapath. However, an easy trick to make a controller is by using a **read-only memory (ROM)** for the combinational part:

ROM outputs = {a_ctrl, b_ctrl, next_state} ← 6 bits

ROM inputs = {state, b_is_0} ← 3 bits



State	b_is_0	Next State	a_ctrl	b_ctrl
S0	x	S1	01 (choose 1)	01 (choose 5)
S1	0	S1	10 (choose A*B)	10 (choose B-1)
S1	1	S2	10 (choose A*B)	10 (choose B-1)
S2	x	S2	00 (choose A)	00 (choose B)

ROM_in	ROM_out (bin)	ROM_out (hex)

37

After we've defined the behavior of the control unit using states and decision logic, the next step is to **synthesize** the actual hardware that implements it. Normally, this involves creating logic circuits for the next-state logic and control signals. However, there's a clever and **simpler alternative** for implementing the **combinational part** of the controller: we can use a **Read-Only Memory (ROM)**.

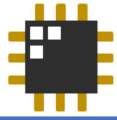
In this method, we **encode all the control outputs and next state transitions** as data entries in the ROM. The **inputs to the ROM** are the **current state** and any **condition flags** (in our case, the b_is_0 flag from the comparator). So, we use **3 input bits**: two for the state (since we only have 3 states, 2 bits are enough) and one for the b_is_1 condition. This gives us a total of $2^3 = 8$ possible input combinations.

The **outputs of the ROM** are the control signals needed for the datapath and the encoded **next state**. Specifically, we need:

- next_state (2 bits to indicate which state to move to)
- a_ctrl (2 bits for A's MUX select), and
- b_ctrl (2 bits for B's MUX select).

That gives us a total of **6 output bits** per ROM entry.

This setup allows us to **store all possible control decisions** in a compact table inside the ROM, which behaves like a lookup table. For any given state and condition flag, the ROM instantly outputs the appropriate MUX controls and next state. This is a fast and reliable way to implement small controllers without having to manually design combinational logic gates for each transition.

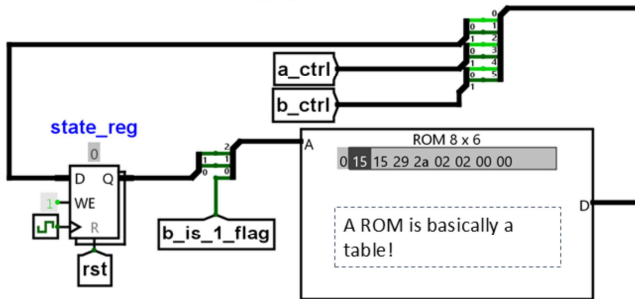


ROM Implementation of Controller

We can do the **usual synthesis** from here to make the controller and connect it to the datapath. However, an easy trick to make a controller is by using a **read-only memory (ROM)** for the combinational part:

ROM outputs = {a_ctrl, b_ctrl, next_state} ← 6 bits

ROM inputs = {state, b_is_0} ← 3 bits



State	b_is_0	Next State	a_ctrl	b_ctrl
S0	x	S1	01 (choose 1)	01 (choose 5)
S1	0	S1	10 (choose A*B)	10 (choose B-1)
S1	1	S2	10 (choose A*B)	10 (choose B-1)
S2	x	S2	00 (choose A)	00 (choose B)

ROM_in	ROM_out (bin)	ROM_out (hex)
00 x	01 01 01	0x15
01 0	10 10 01	0x29
01 1	10 10 10	0x2A
10 x	00 00 10	0x02

Note: ROM input and output bit order can change depending on how the circuit was wired

38

ROM_in: 00 x → ROM_out: 01 01 01 → Hex: 0x15

- Input: State = 00 (S0), b_is_0 = don't care (x)
- Output:
 - a_ctrl = 01 → Selects constant 1 for A
 - b_ctrl = 01 → Selects constant 5 for B
 - next_state = 01 → Move to S1

ROM_in: 01 0 → ROM_out: 10 10 01 → Hex: 0x29

- Input: State = 01 (S1), b_is_0 = 0 (B ≠ 0)
- Output:
 - a_ctrl = 10 → Select A × B
 - b_ctrl = 10 → Select B - 1
 - next_state = 01 → Stay in S1

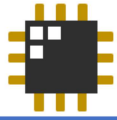
ROM_in: 01 1 → ROM_out: 10 10 10 → Hex: 0x2A

- Input: State = 01 (S1), b_is_0 = 1 (B == 0)
- Output:
 - a_ctrl = 10 → Select A × B
 - b_ctrl = 10 → Select B - 1

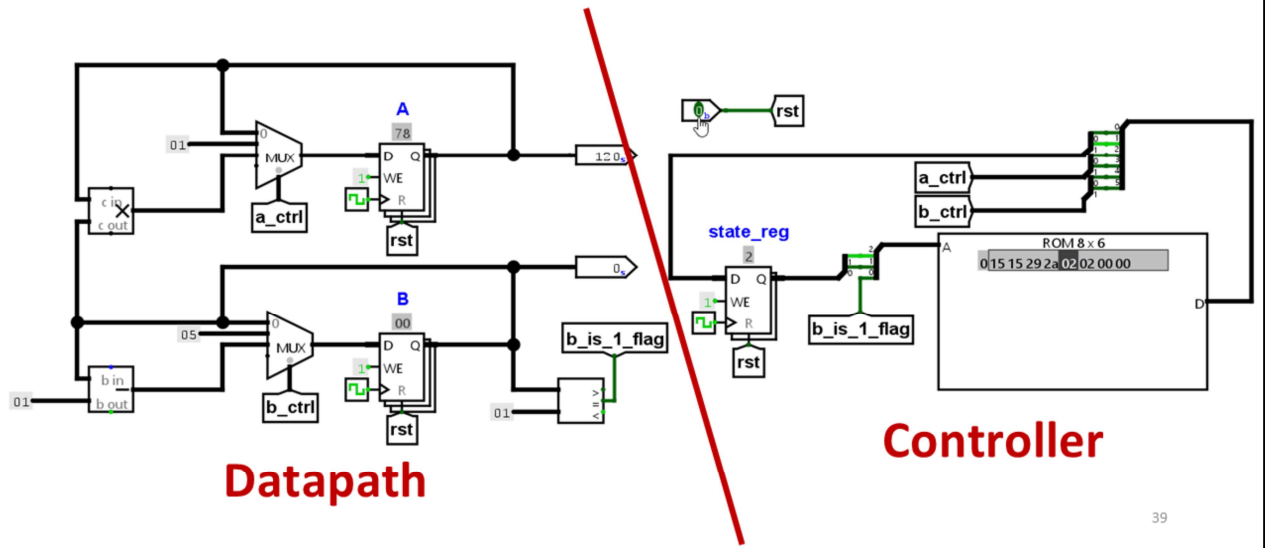
- next_state = 10 → Go to S2

ROM_in: 10 x → ROM_out: 00 00 10 → Hex: 0x02

- Input: State = 10 (S2), b_is_0 = don't care (x)
- Output:
 - a_ctrl = 00 → Keep previous A
 - b_ctrl = 00 → Keep previous B
 - next_state = 10 → Stay in S2



Factorial Example – Final Circuit

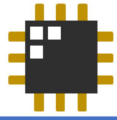


39

The slide presents the **final circuit implementation** of the factorial example, combining both the **datapath** and the **controller** components. The datapath includes the registers A and B, the arithmetic units like the multiplier and subtractor, constants, and multiplexers that select the appropriate inputs to the registers based on control signals.

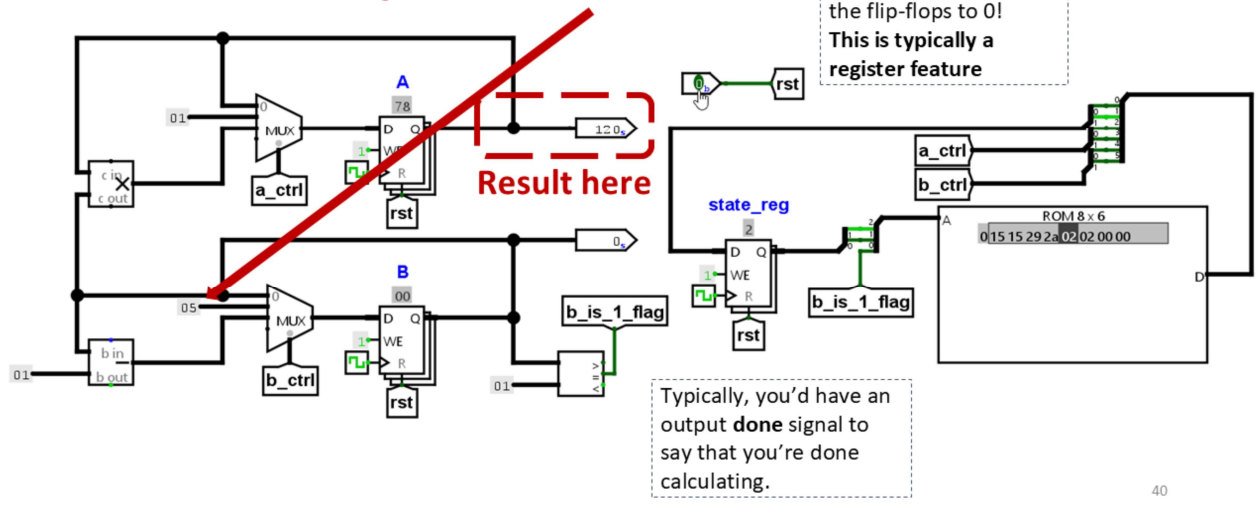
The controller, implemented using the ROM logic we discussed earlier, generates the necessary control signals to dictate the operations performed in each clock cycle. It manages the MUX selections and controls the flow through different states according to the factorial algorithm's needs.

Together, this circuit realizes the ASM design in hardware: the datapath performs the actual data manipulations while the controller sequences these operations according to the states and conditions of the algorithm. This integration showcases how an abstract ASM and RTL description transitions into a practical, working digital circuit.

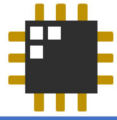


Factorial Example – Final Circuit

This circuit computes for 5!

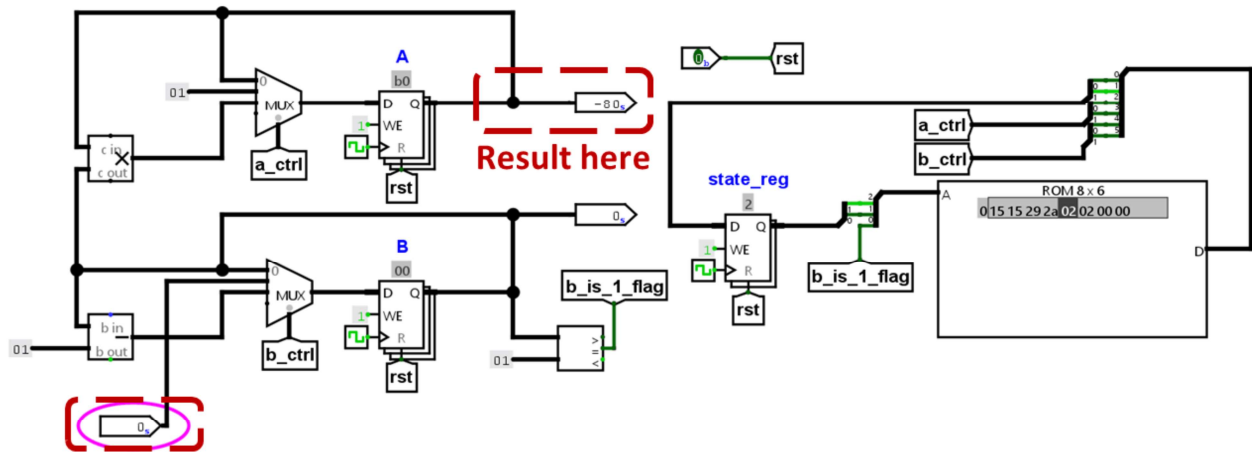


This circuit specifically computes the factorial of 5 (5!). A reset button is included to initialize the registers by setting the flip-flops to zero, which is a common feature in registers to ensure the system starts from a known state. Typically, a complete design would also include an output “done” signal to indicate when the factorial calculation is finished, allowing the system or user to know that the result is ready for use. This signal helps coordinate further operations or data retrieval after computation.



Factorial Example: Generalizability

This circuit computes for n !



We can make the circuit more general by using an input instead of a constant!

In this improved version of the circuit, the factorial calculator now accepts an input value n , instead of being fixed to compute only 5!. This makes the design more flexible and general-purpose, able to calculate the factorial of any valid input within its bit-width limits. The datapath and control logic remain similar, but now the input register for **B** is connected to an external input rather than a constant value. This enhancement moves the circuit closer to a practical, reusable hardware module that can be integrated into larger systems requiring factorial computations.